CONTROL SYSTEM ENGINEERING (ME5659) FINAL PROJECT



LINEARIZATION, MODELING AND CONTROL OF A 2D SUBMARINE MODEL

GROUP MEMBERS:

Mir Munavvar Ali
Logan Buchalter
Savannah Macero
Kevin Shoyer

Background

Submarines represent a series of complex control systems, sensors, and forces that work together to drive the vehicle to a desired pose. Being able to estimate the position of a submarine from onboard sensors is crucial, since the vehicle will often operate underwater/in remote locations that are GPS-denied. Traditionally, submarines in well-mapped environments have relied on PID control to manipulate the rudders, thrusters, and ballast tanks to achieve desired state values [1]. More robust control methods optimize the submarine's performance via the implementation of a cost function [2]. PID controls are limited in that they require the inherently nonlinear system to be linearized. More modern approaches involve adaptive control methods such as sliding mode control and backstepping to create a more robust system that can be directly applied to the nonlinear state space equations [3]. The goal of this project was to consolidate the most important subsystems of a submarine to create a 3-degree-of-freedom state space model. The model allows for two modes of propulsion: a thruster that provides force along the submarine's local x-axis and two differential ballast tanks to manipulate buoyancy and pitch angle. Through different control methods, the submarine's states can be manipulated to specific positions and orientation values.

Vehicle Dynamics

The following parameters, datum, and variables were used in the analysis (Table 1).

Table 1: Parameters and Relevant Values

Description	Unit	Parameter	Value
Acceleration due to gravity	m/s^2	g	9.81
Submarine mass	kg	m	10,000
Specific weight of water	N/m^3	γ	9810
Submarine moment of inertia	$kg \bullet m^3$	I	40,800
Length from center of submarine to ballast tank	m	l_t	3
Volume of forward and aft ballast tanks	m^3	V	0.62
Fractional ratio of air in forward (FT), aft (AT) tanks		P_{FT}, P_{AT}	-0.5 to 0.5
Consolidated drag coefficients in x	kg/m	b_x	615

Consolidated drag coefficients in y	kg/m	b_y	1960
Consolidated drag coefficients in θ	kg/m	b_θ	10,000
Thrust force	N	F_{thrust}	
Ballast force forward (FT), aft (AT)	N	F_{FT}, F_{AT}	
Drag force in x, y	N	F_{Dx}, F_{Dy}	
Drag torque	N•m	$T_{D\theta}$	
Position in x,y	m	x, y	
Velocity in x,y	m/s	$\dot{x}$, $\dot{y}$	
Acceleration in x,y	m/s^2	$\ddot{x}$, $\ddot{y}$	
Pitch angle	°	θ	
Angular velocity	°/s	$\dot{\theta}$	
Angular acceleration	°/s^2	$\ddot{\theta}$	

The derivation of the system began with creating a free body diagram as dictated by the project scope. Figure 1 is the FBD created.

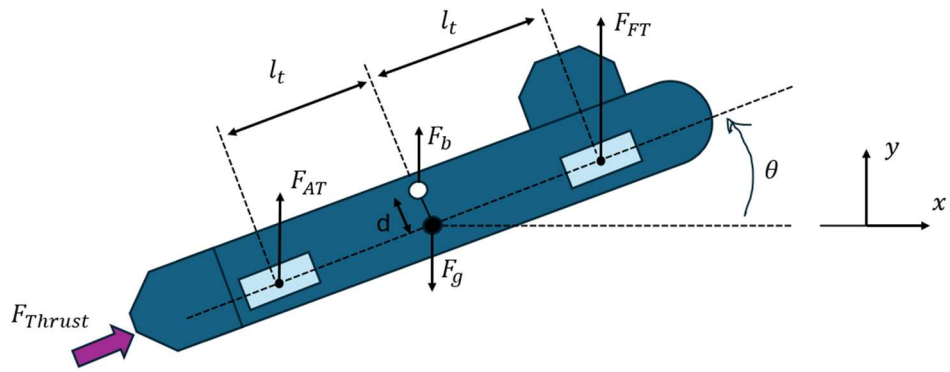


Figure 1: Free body diagram of submarine model

Based on Figure 1, three primary equations were derived for the y acceleration, x acceleration, and angular acceleration (in x-y plane), shown below in order.

$$\sum F_y = F_{thrust} sin\theta - mg + F_b + F_{AT} + F_{FT} = m\ddot{y} \quad [1]$$

$$\sum F_x = F_{thrust} cos\theta = m\ddot{x} \quad [2]$$

$$\sum T = F_{FT} \times l_t - F_{AT} \times l_t - F_b \times d = (F_{FT} - F_{AT}) l_t cos\theta - F_b d sin\theta = I\ddot{\theta} \quad [3]$$

The total buoyant force F_b of the submarine itself was assumed to be the same as gravity, and the offset d between the center of buoyancy and center of gravity was deemed minimal. Additionally, the expressions for drag force/torque were consolidated and simplified, as seen in Equations 4-6.

$$F_{Dx} = -\frac{1}{2}\rho C_{d,x} A_x |\dot{x}|\dot{x} = -b_x(|\dot{x}|)\dot{x} \quad [4]$$

$$F_{Dy} = -\frac{1}{2}\rho C_{d,y} A_y |\dot{y}|\dot{y} = -b_y(|\dot{y}|)\dot{y} \quad [5]$$

$$T_{D\theta} = -\frac{1}{2}\rho C_{d,\theta} A_\theta |\dot{\theta}|\dot{\theta} = -b_\theta(|\dot{y}|)\dot{y} \quad [6]$$

F_{FT} and F_{AT} were substituted as seen Equations 7-8, where P_{FT} and P_{AT} are system inputs to influence buoyant force and torque.

$$F_{FT} = \gamma V P_{FT} \quad [7]$$

$$F_{AT} = \gamma V P_{AT} \quad [8]$$

Substituting the buoyant force, drag force, and cancelling system buoyancy with gravity, the final equations of motion were developed:

$$\sum F_y = F_{thrust} sin\theta + \gamma V(P_{AT} + P_{FT}) - b_y|\dot{y}|\dot{y} = m\ddot{y} \quad [9]$$

$$\sum F_x = F_{thrust} cos\theta - b_x\dot{x} = m\ddot{x} \quad [10]$$

$$\sum T = \gamma V(P_{FT} - P_{AT}) l_t cos\theta - b_\theta|\dot{\theta}|\dot{\theta} = I\ddot{\theta} \quad [11]$$

The state vector and input vector are selected to be:

$$x = \left[x\ y\ \theta\ \dot{x}\ \dot{y}\ \dot{\theta}\right]^T = [x_1\ x_2\ x_3\ x_4\ x_5\ x_6]^T \quad [12]$$

$$u = [P_{AT}\ P_{FT}\ F_t]^T = [u_1\ u_2\ u_3]^T \quad [13]$$

The non-linear dynamics equation is:

$$\dot{x} = \begin{bmatrix} x_4 \\ x_5 \\ x_6 \\ \frac{1}{m}(F_{thrust}\cos(x_3) - b_x x_4|x_4|) \\ \frac{1}{m}(F_{thrust}\sin(x_3) + \gamma V(P_{AT} + P_{FT}) - b_y x_5|x_5|) \\ \frac{1}{I}(\gamma V(P_{FT} - P_{AT}) l_t cos(x_3) - b_\theta x_6|x_6|) \end{bmatrix} \quad [14]$$

The system is linearized about the nominal operating state, $\tilde{x}$ and nominal input, $\tilde{u}$.

$$\dot{x} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{-1}{m}(\tilde{u}_3 \sin(\tilde{x}_3)) & \frac{-2b_x}{m}\tilde{x}_4 & 0 & 0 \\ 0 & 0 & \frac{1}{m}(F_{thrus}\ cos(\tilde{x}_3)) & 0 & \frac{-2b_y}{m}\tilde{x}_5 & 0 \\ 0 & 0 & \frac{-1}{I}(\gamma V(\tilde{u}_2 - \tilde{u}_1) l_t sin(\tilde{x}_3) & 0 & 0 & \frac{-2b_\theta}{I}\tilde{x}_6 \end{bmatrix} x + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{m}\cos(\tilde{x}_3) \\ \frac{1}{m}\gamma V & \frac{1}{m}\gamma V & \frac{1}{m}\sin(\tilde{x}_3) \\ \frac{-1}{I}\gamma V l_t cos(\tilde{x}_3) & \frac{1}{I}\gamma V l_t cos(\tilde{x}_3) & 0 \end{bmatrix} u$$

For an arbitrary nonzero input, our open loop system realization is not asymptotically stable. Using the PBH rank test, we found that the system at rest is stable in the sense of Lyapunov. Thus, we linearize the nonlinear dynamics about $\tilde{x} = 0$ and $\tilde{u} = 0$, corresponding to the submarine at rest. The state feedback gain K places desired eigenvalues such that the closed loop system is asymptotically stable. As discussed below, although the linear model will diverge from the non-linear dynamics during motion, this selected operating point is sufficient for controller design.

Controllability and Observability

One can compute the Controllability matrix of the MIMO system above by

$$P = [B\ AB\ A^2B\ \dots A^{n-1}B]$$

P is full rank, regardless of selection of operating point and therefore, the system is controllable. The observability of the system is dependent on the selection of the C matrix. For the system to be observable, it is sufficient to have $C = [I_{3x3}\ 0_{3x3}]$, meaning the output y is simply $y = [x, y, \theta]^T$.

CONTROLLER DESIGN

CONTROLLER 1: STATE FEEDBACK WITH STEADY-STATE TRACKING

Steady-state tracking and state feedback control may be implemented using the closed-loop feedback equations:

$$u(t) = -Kx(t) + Gr(t)$$

$$\dot{x} = (A - BK)x(t) + Br(t)$$

Where $r(t)$ is the desired output $[x_d, y_d, \theta_d]^T$. The Gains K and G are designed to ensure desired performance. We desire a 1% maximum overshoot and a settling time of 60 seconds. This yields the

desired closed-loop eigenvalues $\lambda_{1,2} = -0.0667 \pm 0.0455i$. Note that our system is MIMO, meaning the Bass-Gura and Ackermann formula cannot be readily applied to calculate K. The "place()" command in MATLAB is utilized which can handle MIMO systems. In SISO systems, it is typical to place other poles sufficiently left of the dominant poles such that the system acts as a second-order system. However, although this achieved an asymptotically stable system, the θ state was not sufficiently controlled to prevent aliasing in the linearized model. The following desired poles were selected to achieve desired performance: $\lambda = [\lambda_1, \lambda_2, \lambda_1, \lambda_2, 1.1\lambda_1, 1.1\lambda_2]$. $K = place(A, B, \lambda)$.

The G matrix may be calculated for the MIMO system using pseudo-inversion:

$$G = -[C(A - B * K)^{-1}B]^T(C(A - BK)^{-1}B[C(A - BK)^{-1}B]^T)^{-1}$$

With the controller designed, suppose the vehicle begins at an initial state: $x(0) = [0\ 0\ 0\ 0\ 0\ 0]^T$ and the desired input is selected to be a step input: $r = [100\ 20\ 0]^T$. The output of the linearized system is modeled using the "lsim()" command in MATLAB. Using this same controller designed around the linearized plant, "ode45()" command is used to simulate the non-linear plant dynamics. These results are shown in Figure 2 below.

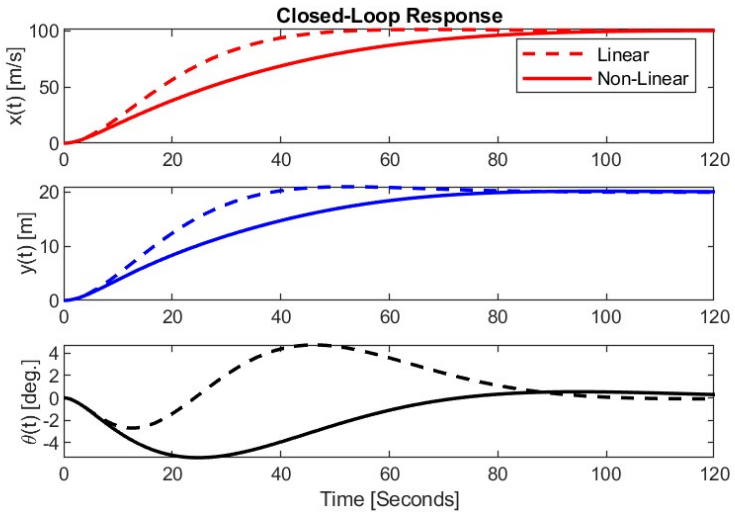


Figure 2: Linear vs nonlinear plant dynamics, state feedback

Despite the large linearization assumptions for controller design, the non-linear system still has satisfactory response (i.e. stability, steady-state value, settling time, overshoot). As expected, the nonlinear model is more damped and takes longer to reach the desired state due to drag increasing with velocity. A second interesting metric is the controller effort, defined as percent of the maximum possible input. This is plotted below in Figure 3.

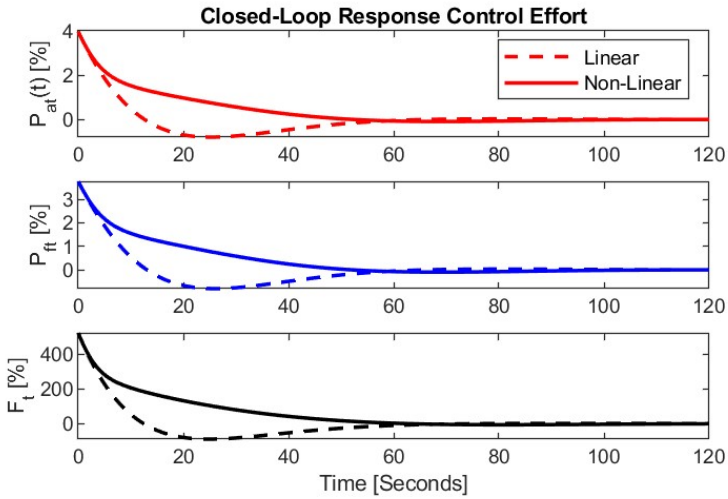


Figure 3: Linear vs nonlinear controller effort

Note that Ft exceeds 100%. This means that it would take more thrust than the motor can generate. A couple of mitigations could be made. From a modeling perspective, a saturation value could be modeled such that 100% is not exceeded. From a controller design perspective, one could adjust the poles of the of the system to satisfy this condition, if desired. Finally, from a plant design perspective, one could specify motors with greater torque if this response is necessary. Finally, the tank fill is small compared to the maximum allowable value meaning the system could dive faster if desired or the tanks could be replaced with smaller tanks.

CONTROLLER 2: OBSERVER-BASED COMPENSATOR

Using the state feedback controller as a template, an observer-based system was created to view states in real time. Observer eigenvalues were selected such that the poles were located 10 times further left than the controller values. The resulting observer values are reported as:

$$L = \begin{bmatrix} 1.467 & 0 & 0 \\ 0 & 1.333 & 0 \\ 0 & 0 & 1.333 \\ 0.788 & 0 & 0 \\ 0 & 0.651 & 0 \\ 0 & 0 & 0.651 \end{bmatrix}$$

This L matrix was combined with the previously computed K to form a new observer-based compensator of size 12x12. The closed-loop state space equations are seen below.

$$u(t) = -K\hat{x}(t) + Gr(t)$$

$$\dot{\hat{x}}(t) = (A - BK - LC)\hat{x}(t) + BGr(t) + LCx(t)$$

With arbitrary starting points for the state observations, the observer was able to track the actual state values, as seen in Figure 4 below. In this example, the starting estimates for [x,y,ϑ] were [15,-10,12], while the target values were set to [20,-14,3].

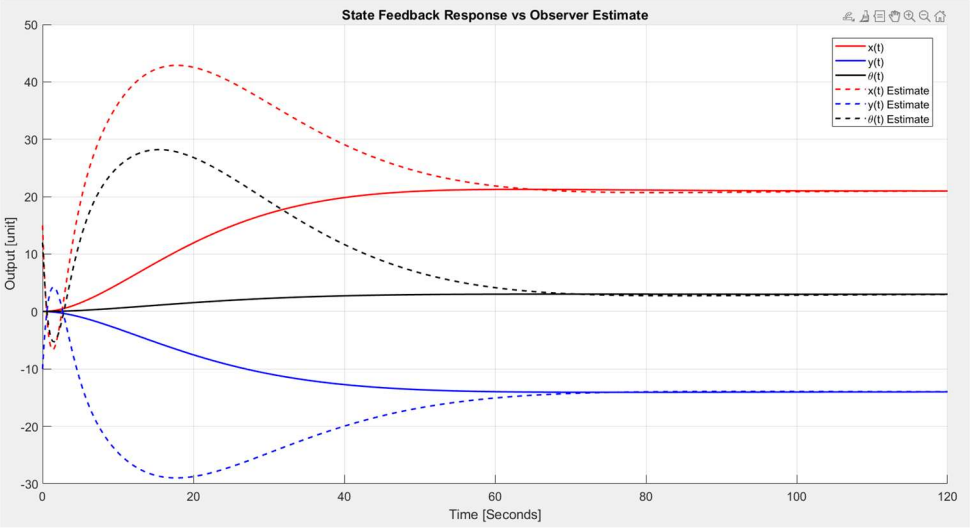


Figure 4: State estimation vs actual state, observer-based compensator

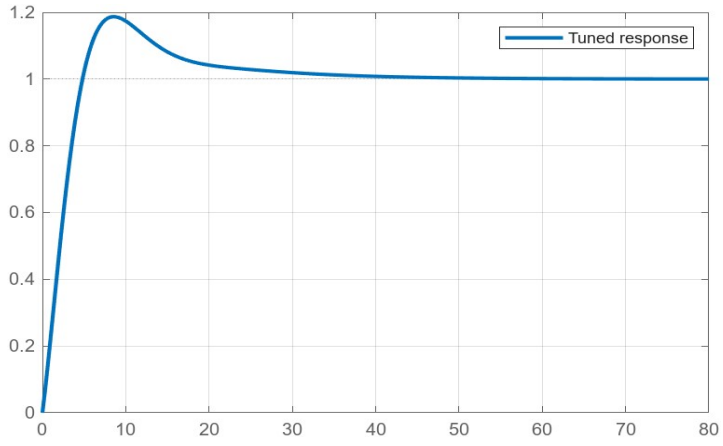
CONTROLLER 3: PID

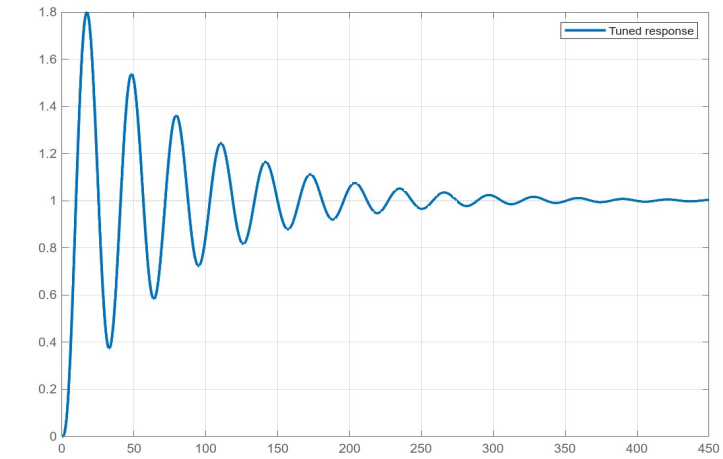
The third controller utilizes the concept of classical PID. The physical parameters of the submarine were the same as the previous designs, and again the system was linearized about the zero equilibrium point to obtain the A and B matrices. Transfer functions for position (x), depth (z), and orientation (θ) were derived from the linearized model. PID gains were computed using Ziegler–Nichols tuning formulas, and unity feedback loops were implemented to form the closed-loop systems. The system's response to step inputs was simulated to evaluate performance, and MATLAB's PID Tuner was integrated to allow fine-tuning of controller gains. This approach demonstrates an alternative approach to the control problem using classical control techniques for multi-variable systems. Figures 5-7 below show the unit step response for [x,y,ϑ] with initial conditions [0,0,0].

Table 2: PID Controller Parameters

PID_Controller_Parameters

Plant	Kp	Ki	Kd	Rise Time (s)	Settling Time (s)
theta	-0.00063511	-0.000000060546	0	3870	28400
x	0.00094081	0.0000016241	0.13625	5.7	300
y	0.053498	0.0015132	0.073343	9.37	80.1





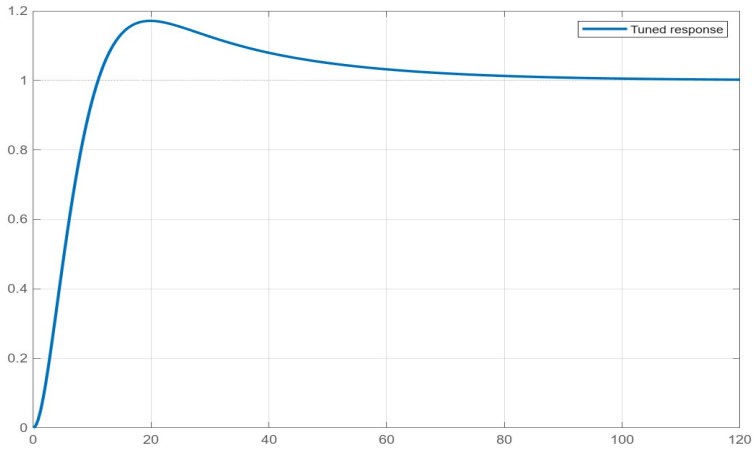


Figure 5 (top, x), Figure 6 (middle, y), Figure 7 (bottom, ϑ): Step response of PID-controlled system

RESULTS

The main decisions that governed the project were related to the modeling of the system. First, the inherent buoyancy force of the submarine was equated to the force of gravity, while the drag parameters in each direction were consolidated into a single term. The system was also linearized about $\tilde{x} = 0$, which assumed that deviations in the submarine pitch were minimal. A key challenge in the design process was choosing an input to linearize about. We chose to design the controller to drive the system to a target position, i.e. $x_d = [x_1\ x_2\ x_3]^T$. We posited that given the dynamics of our state equation, the linearized state equation about the zero state and zero input is a suitable model for the desired nonlinear closed loop response. While we found our controller to both asymptotically stabilize

the closed loop system and achieve the desired transient behavior, further experimentation with desired output states revealed that our system realization cannot directly drive the velocity terms to a desired steady state value. Next steps could involve formulating a more robust realization that would provide more flexibility or choosing a different state vector that enables direct velocity control. A key takeaway was that the facets of control design are much more complex than we initially thought, but as a result allows for creativity and style. By applying the concepts from class in a specific context we gained a better understanding of the motivation and purpose of the coursework.

References

[1] Fossen, Thor. *Handbook of Marine Craft Hydrodynamics and Motion Control*. *Wiley.com*, 2024, www.wiley.com/en-be/Handbook+of+Marine+Craft+Hydrodynamics+and+Motion+Control-p-9781119994138. Accessed 13 Apr. 2025.

[2] von Ellenrieder, Karl Dietrich. *Control of Marine Vehicles*. *Springer Series on Naval Architecture, Marine Engineering, Shipbuilding and Shipping*, Cham, Springer International Publishing, 2021.

[3] Yuh, J. "Design and Control of Autonomous Underwater Robots: A Survey." *Autonomous Robots*, vol. 8, no. 1, Jan. 2000, pp. 7–24, https://doi.org/10.1023/a:1008984701078.

[4] MathWorks. (n.d.). *PID controller design using MATLAB and Simulink*. https://www.mathworks.com/help/control/ug/pid-controller-design.htmlhttps://www.mathworks.com/help/control/ug/pid-controller-design.html

APPENDIX A: GROUP CONTRIBUTIONS

The group collectively met and worked to model the system, derive the state space equations, define project objectives, and brainstorm solutions. Savannah and Kevin worked primarily on the state feedback controller. Logan worked on the observer-based controller, and Munavvar Ali worked on the PID controller. The work was split evenly between all group members.

APPENDIX B: MATLAB CODE

APPENDIX B1: MATLAB CODE – STATE FEEDBACK CONTROLLER

```
%clc;
%clear;

% Intputs: Where does the vehicle start and where do you want to go?
rd = [100 20 0]'; %desired state at tf [xd,yd,thetad]
x0 = [0 0 0 0 0 0]; %initial conditions [x,y,theta,vx,vy,w]

%vehicle dynamic constants
rho = 1000; %denisty of water
g = 8.91; %gravity in m/s^2
D = 2.8; %width of sub
m = 10000; %mass of vehicle
L = 7; %length of vehicle
I = 1/12*m*L^2;
l = 3;
Cd = 0.2; %drag coefficient
bx = 1/2*Cd*(pi*(D)^2/4)*rho;
by = 1/2*Cd*(D*L)*rho;
bth = 10000; %assuming ~10x that of by
V = L*(pi*(D)^2/4); %total volume
Vt = 1/10*V;
Vgam = rho*g*Vt;

%% terminal velocity calculations
Pat_max = .5; %max fill of aft tank
Pft_max = .5; %max fill of forward tank
Fft_max = 1385.4; %max thrust newtons
vx_terminal = sqrt(Fft_max/bx);
vy_terminal = sqrt(Vgam*(Pat_max+Pft_max)/by);

%% State space linearization
%state to linearize about
x1e = 0;
x2e = 0;
x3e = 0;
x6e = 0;
x4e = 0;
x5e = 0;
%if linearizing about terminal velocities
```

```
%x4e = 1.5;
%x5e = -6.26;
%if we linearize about 1/4 of terminal velocities
% x4e = 1.5/4;
% x5e = 6.26/4;
% if we linearize about 0

xe = [x1e x2e x3e x4e x5e x6e]';

%inputs to linearize about
Pat = 0;
Pft = 0;
Ft =  0;
%if we linearize about maximum inputs
% Pat = -0.5;
% Pft = -0.5;
% Ft =  1385.4;
%if we linearize about half of maximum inputs
% Pat = -0.5/2;
% Pft = -0.5/2;
% Ft =  1385.4/2;

%% Compute Linearized A and B matrix
A = [0 0 0 1 0 0;
    0 0 0 0 1 0;
    0 0 0 0 0 1;
    0 0 -(1/m)*Ft*sin(x3e) -(2/m)*bx*abs(x4e) 0 0;
    0 0 (1/m)*Ft*cos(x3e) 0 -(2/m)*by*abs(x5e) 0;
    0 0 (-1/I)*(-Pat + Pft)*Vgam*l*sin(x3e) 0 0 (-2/I)*bth*abs(x6e)]; %confirmed this is
correct through taking jacobian of symbolic eqution s

B = [0 0 0;
    0 0 0;
    0 0 0;
    0 0 (1/m)*cos(x3e);
    (1/m)*Vgam (1/m)*Vgam (1/m)*sin(x3e);
    (-1/I)*Vgam*l*cos(x3e) (1/I)*Vgam*l*cos(x3e) 0];

%% Check for controllability
P=ctrb(A,B);
rank(P);

%% Check for observability
C = eye(6);
C = C(1:3,:);
D = zeros(3,3);
Q = obsv(A,C);
rank(Q);

%% Build open loop system
sys0 = ss(A,B,C,D); %open loop system

%% State feedback controller with steady state tracking

%doing it the method we learned for SISO
% POd = 1; %desired percent overshoot [%]
% tsd = 60; %desired settling time
```

```
%
% zd = abs(log(POd/100))/sqrt(pi^2+log(POd/100)^2); %desired zeta
% wnd = 4/(zd*tsd); %desired wn
%
% dend = [1 2*zd*wnd wnd^2]; %desired denominator coefficients for transfer function
% eigd = roots(dend); %desired roots
% % eigd = [-1 -2]';
% p = [eigd',10*real(eigd(1)),11*real(eigd(1)),12*real(eigd(1)),13*real(eigd(1))]; %de-
sired poles
%% trying to desing pole values around x,y,theta independently
% this doesnt work as we want it to
% POd = [1; 1; 1]; %desired percent overshoot [%]
% tsd = [120; 60; 10000]; %desired settling time
%
% zd = abs(log(POd/100))./sqrt(pi^2+log(POd/100).^2); %desired zeta
% wnd = 4./(zd.*tsd); %desired wn
% dend = [[1;1;1] 2.*zd.*wnd wnd.^2];
% p = [roots(dend(1,:))';roots(dend(2,:))';roots(dend(3,:))']

%% ITAE characteristic Polynomial approach
% Alternatively, using the ITAE characteristic polynomial (6th order)
 % dend = [1 3.25*wnd 6.6*wnd^2 8.6*wnd^3 7.45*wnd^4 3.95*wnd^5 wnd^6];
 % p = roots(dend)

%% repeating the desired roots:
POd = 1; %desired percent overshoot [%]
tsd = 60; %desired settling time

zd = abs(log(POd/100))/sqrt(pi^2+log(POd/100)^2); %desired zeta
wnd = 4/(zd*tsd); %desired wn

dend = [1 2*zd*wnd wnd^2]; %desired denominator coefficients for transfer function
eigd = roots(dend); %desired roots
p = [eigd';eigd';1.1*eigd'];

K = place(A,B,p);
G = -[C*inv(A-B*K)*B]'*inv(C*inv(A-B*K)*B*[C*inv(A-B*K)*B]')
Acl = A-B*K;
Bcl = B*G;
sys1 = ss(Acl,Bcl,C,D);

t0 = 0; %initial time (s)
tfin = 120; %final time (s)
dt = .1;
tv = [t0:dt:tfin]'; %time vector for lsim
% rd = [-.5 -.5  1385.4]';
rv = ones(length(rd),length(tv)).*rd;
rv = rv';
[Yc1,t,Xc1] = lsim(sys1,rv,tv,x0); % Close-loop response
[Yo,t,Xo] = lsim(sys0,rv,tv,x0); %
X1 = Xo;
%X1 = xe'+Xo; %the actual state is the output of the sim + expected state
figure(1);
clf
hold on
%plot(t,Yo,'lineWidth',1.5,'displayName','x(t) Open-Loop Output')
```

```
plot(t,Yo(:,1)./Yo(end,1),'r--','lineWidth',1.5,'displayName','x(t) open-loop (normal-
ized)')
hold on
plot(t,Yo(:,2)./Yo(end,2),'b--','lineWidth',1.5,'displayName','y(t) open-loop')
plot(t,Yo(:,3)./Yo(end,3),'k--','lineWidth',1.5,'displayName','\theta(t) open-loop')
plot(t,Yc1(:,1)./max(abs(Yc1(:,1))),'r','lineWidth',1.5,'displayName','x(t) Closed-
loop')
maxvals = max(Yc1,[],1)
text1 = ['max values are [x(t),y(t),theta(t)] = [' ,num2str(maxvals),']']
text(.1,-.1,text1)
hold on
plot(t,Yc1(:,2)./max(abs(Yc1(:,2))),'b','lineWidth',1.5,'displayName','y(t) Closed-
loop')
plot(t,Yc1(:,3)./max(abs(Yc1(:,3))),'k','lineWidth',1.5,'displayName','\theta(t) Closed-
loop')
grid;
set(gca,'FontSize',12);
title('Open-Loop and Closed-Loop Step Response')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')

% figure(1);
% clf
% hold on
%plot(t,Yo,'lineWidth',1.5,'displayName','x(t) Open-Loop Output')
% plot(t,Yo(:,1),'r--','lineWidth',1.5,'displayName','x(t) open-loop (normalized)')
% hold on
% plot(t,Yo(:,2),'b--','lineWidth',1.5,'displayName','y(t) open-loop')
% plot(t,Yo(:,3),'k--','lineWidth',1.5,'displayName','\theta(t) open-loop')
% plot(t,Yc1(:,1),'r','lineWidth',1.5,'displayName','x(t) Closed-loop')
% hold on
% plot(t,Yc1(:,2),'b','lineWidth',1.5,'displayName','y(t) Closed-loop')
% plot(t,Yc1(:,3),'k','lineWidth',1.5,'displayName','\theta(t) Closed-loop')
% grid;
% set(gca,'FontSize',12);
% title('Open-Loop and Closed-Loop Step Response')
% ylabel('Output [unit]')
% xlabel('Time [Seconds]')
% legend show
% legend('location','best')

%%

tspan = [t0 tfin];
[t2,X] = ode45(@(t,X)xdot_nonlin(t,X,tv,rv),tspan,x0);
[t3,X2] = ode45(@(t,X)xdot_lin(t,X,tv,rv),tspan,x0);

% closed loop response of non-linear system
[t4,X4] = ode45(@(t,X)xdot_nonlin_closedloop(t,X,tv,rv,K,G),tspan,x0);

%% Open loop linearized vs non-linear comparison

% figure(4);
% clf
%linear model using ODE45 - This gives exact same results as LSIM for the
```

```
%linear system, used to check implementation
% plot(t3,X2(:,1),'ro','lineWidth',1.5,'displayName','x(t) Linearized ODE45')
% hold on
% plot(t3,X2(:,2),'bo','lineWidth',1.5,'displayName','y(t) Linearized ODE45')
% plot(t3,X2(:,3),'ko','lineWidth',1.5,'displayName','\theta(t) Linearized ODE45')
% Linear model Using LSIM
% plot(t,X1(:,1),'r','lineWidth',1.5,'displayName','x(t) Linearized LSIM')
% hold on
% plot(t,X1(:,2),'b','lineWidth',1.5,'displayName','y(t) Linearized LSIM')
% plot(t,X1(:,3),'k','lineWidth',1.5,'displayName','\theta(t) Linearized LSIM')
% %non-linear model
% plot(t2,X(:,1),'r--','lineWidth',1.5,'displayName','x(t) Non-linear')
% plot(t2,X(:,2),'b--','lineWidth',1.5,'displayName','y(t) Non-linear')
% plot(t2,X(:,3),'k--','lineWidth',1.5,'displayName','\theta(t) Non-linear')
% grid;
% set(gca,'FontSize',12);
% title('Linear vs non-linear open loop Step Response')
% ylabel('Output [unit]')
% xlabel('Time [Seconds]')
% legend show
% legend('location','best')
% %% velocity
% figure(5);
% plot(t,X1(:,4),'r','lineWidth',1.5,'displayName','x(t) Linearized')
% hold on
% plot(t,X1(:,5),'b','lineWidth',1.5,'displayName','y(t) Linearized')
% plot(t,X1(:,6),'k','lineWidth',1.5,'displayName','\theta(t) Linearized')
% plot(t2,X(:,4),'r--','lineWidth',1.5,'displayName','x(t) Non-linear')
% plot(t2,X(:,5),'b--','lineWidth',1.5,'displayName','y(t) Non-linear')
% plot(t2,X(:,6),'k--','lineWidth',1.5,'displayName','\theta(t) Non-linear')
% grid;
% set(gca,'FontSize',12);
% title('Linear vs non-linear open loop Step Response')
% ylabel('Output [unit]')
% xlabel('Time [Seconds]')
% legend show
% legend('location','best')

%compare closed loop for linear model vs non-linear model
%%
figure(5);
clf
hold on
plot(t,Yc1(:,1),'r--','lineWidth',1.5,'displayName','x(t) Linear')
plot(t,Yc1(:,2),'b--','lineWidth',1.5,'displayName','y(t) Linear')
plot(t,Yc1(:,3),'k--','lineWidth',1.5,'displayName','\theta(t) Linear')
plot(t4,X4(:,1),'r','lineWidth',1.5,'displayName','x(t) Non-Linear')
plot(t4,X4(:,2),'b','lineWidth',1.5,'displayName','y(t) Non-Linear')
plot(t4,X4(:,3),'k','lineWidth',1.5,'displayName','\theta(t) Non-Linear')
grid;
set(gca,'FontSize',12);
title('Closed-Loop Response for Linear model vs non-linear model')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')
%%
```

```
figure(6)
clf
hold on
plot(t,Xc1(:,4),'r--','lineWidth',1.5,'displayName','$\dot{x}(t)$ Linear')
plot(t,Xc1(:,5),'b--','lineWidth',1.5,'displayName','$\dot{y}(t)$ Linear')
plot(t,Xc1(:,6),'k--','lineWidth',1.5,'displayName','$\dot{\theta}(t)$ Linear')
plot(t4,X4(:,4),'r','lineWidth',1.5,'displayName','$\dot{x}(t)$ Non-Linear')
plot(t4,X4(:,5),'b','lineWidth',1.5,'displayName','$\dot{y}(t)$ Non-Linear')
plot(t4,X4(:,6),'k','lineWidth',1.5,'displayName','$\dot{\theta}(t)$ Non-Linear')
grid;
set(gca,'FontSize',12);
title('Closed-Loop Response for Linear model vs non-linear model')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')
legend('Interpreter',"latex")

%% Controller
% Recompute the controller values used in the non-linear model:
% time vectors used for controler input and output of ode45 are not
% consistant. interpolate:

%linear controller values
rv1 = zeros(3,length(t));
rv1(1,:) = pchip(tv,rv(:,1),t);
rv1(2,:) = pchip(tv,rv(:,2),t);
rv1(3,:) = pchip(tv,rv(:,3),t);
U_controller_lin = -K*Xc1'+G*rv1;

%nonlinear controller values
rv4 = zeros(3,length(t4));
rv4(1,:) = pchip(tv,rv(:,1),t4);
rv4(2,:) = pchip(tv,rv(:,2),t4);
rv4(3,:) = pchip(tv,rv(:,3),t4);
U_controller_nonlin = -K*X4'+G*rv4;

figure(7) %controller inputs
Pat_max = .5; %max fill of aft tank
Pft_max = .5; %max fill of forward tank
Fft_max = 1385.4; %max thrust newtons
plot(t,U_controller_lin(1,:)./Pat_max,'r--','displayName','P_{at} Linear')
hold on
plot(t,U_controller_lin(2,:)./Pft_max,'b--','displayName','P_{ft} Linear')
plot(t,U_controller_lin(3,:)./Fft_max,'k--','displayName','F_t Linear')
plot(t4,U_controller_nonlin(1,:)./Pat_max,'r','displayName','P_{at} Non-Linear')
plot(t4,U_controller_nonlin(2,:)./Pft_max,'b','displayName','P_{ft} Non-Linear')
plot(t4,U_controller_nonlin(3,:)./Fft_max,'k','displayName','F_t Non-Linear')
title("Normalized Control Effort for linear vs non-linear model")
legend show
ylabel('Normalized Control Effort')
xlabel('time (s)')

%% What at if each time step, we recompute the A and B matrix?
tspan1 = [0:.001:120];
[t5,X5] = ode45(@(t,X)xdot_lin_withUpdates(t,X,tv,rv,K,G),tspan1,x0);
```

```
figure(10);
clf
hold on
plot(t5,X5(:,1),'r--','lineWidth',1.5,'displayName','x(t) Linear w/ updates')
plot(t5,X5(:,2),'b--','lineWidth',1.5,'displayName','y(t) Linear w/ updates')
plot(t5,X5(:,3),'k--','lineWidth',1.5,'displayName','\theta(t) Linear w/ updates')
plot(t4,X4(:,1),'r','lineWidth',1.5,'displayName','x(t) Non-Linear')
plot(t4,X4(:,2),'b','lineWidth',1.5,'displayName','y(t) Non-Linear')
plot(t4,X4(:,3),'k','lineWidth',1.5,'displayName','\theta(t) Non-Linear')
grid;
set(gca,'FontSize',12);
title('Response for Linear w/updates model vs non-linear model')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')
%%
figure(11)
clf
hold on
plot(t5,X5(:,4),'r--','lineWidth',1.5,'displayName','$\dot{x}(t)$ Linear w/ updates')
plot(t5,X5(:,5),'b--','lineWidth',1.5,'displayName','$\dot{y}(t)$ Linear w/ updates')
plot(t5,X5(:,6),'k--','lineWidth',1.5,'displayName','$\dot{\theta}(t)$ Linear w/ up-
dates')
plot(t4,X4(:,4),'r','lineWidth',1.5,'displayName','$\dot{x}(t)$ Non-Linear')
plot(t4,X4(:,5),'b','lineWidth',1.5,'displayName','$\dot{y}(t)$ Non-Linear')
plot(t4,X4(:,6),'k','lineWidth',1.5,'displayName','$\dot{\theta}(t)$ Non-Linear')
grid;
set(gca,'FontSize',12);
title('Response for Linear w/ updates model vs non-linear model')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')
legend('Interpreter',"latex")
```

APPENDIX B2: MATLAB CODE – OBSERVER BASED CONTROLLER

```
%% Workspace Variables in this script are initialized identically to B1, omitted here
for readability%%
%Observer
pL = 10*p;

L = place(A',C',pL)';
AL = [A -B*K; L*C A-B*K-L*C];
BL = [B*G; B*G];
CL = [C zeros(size(C))];
DL = 0;
x0L = [x0 15 -10 12 -18 11 -13];
```

```
sys2 = ss(AL,BL,CL,DL);

t0 = 0; %initial time (s)
tfin = 120; %final time (s)
dt = .1;
tv = [t0:dt:tfin]'; %time vector for lsim
% rd = [-.5 -.5  1385.4]';
rv = ones(length(rd),length(tv)).*rd;
rv = rv';
[Yc1,t,Xc1] = lsim(sys1,rv,tv,x0); % Close-loop response
[Yo,t,Xo] = lsim(sys0,rv,tv,x0); %
[YL,t,XL] = lsim(sys2,rv,tv,x0L);

figure(5);
clf
hold on
plot(t,Yc1(:,1),'r','lineWidth',1.5,'displayName','x(t)')
plot(t,Yc1(:,2),'b','lineWidth',1.5,'displayName','y(t)')
plot(t,Yc1(:,3),'k','lineWidth',1.5,'displayName','\theta(t)')
plot(t,XL(:,7),'r--','lineWidth',1.5,'displayName','x(t) Estimate')
plot(t,XL(:,8),'b--','lineWidth',1.5,'displayName','y(t) Estimate')
plot(t,XL(:,9),'k--','lineWidth',1.5,'displayName','\theta(t) Estimate')
grid;
set(gca,'FontSize',12);
title('State Feedback Response vs Observer Estimate')
ylabel('Output [unit]')
xlabel('Time [Seconds]')
legend show
legend('location','best')

%% Animation
figure;
fig = figure('Position', [100, 100, 1280, 720]);  % Or 1920x1080 for Full HD
x = Yc1(1:500,1);
y = -Yc1(1:500,2);
theta = -Yc1(1:500,3);
% Load and show background image
bg = imread('ocean_background.jpg');
x_range = [min(x)-30, max(x)+30];
y_range = [min(y)-30, max(y)+10];

image('XData', x_range, 'YData', y_range, 'CData', bg);
set(gca, 'YDir', 'normal');

hold on;
axis equal;
xlim(x_range);
ylim(y_range);
% Load and prepare image
[sub_img, ~, alpha] = imread('titan.png');
sub_img = im2double(sub_img);
hasAlpha = ~isempty(alpha);

% Set up figure
% Create a VideoWriter object
video = VideoWriter('submarine_animation2', 'MPEG-4');  % 'avi' or 'MP4'
```

```
video.FrameRate = 30;  % Control playback speed (frames per second)
open(video);  % Open video file for writing

% Set up figure and axis (similar to before)
trail = plot(NaN, NaN, 'r', 'LineWidth', 1.5);  % Trail path

% Set axis limits (buffer for image size)
buffer = 30;
xlim([min(x) - buffer, max(x) + buffer]);
ylim([min(y) - buffer, max(y) + buffer-20]);

% Image dimensions in plot units
sub_w = 30;
sub_h = 15;

% Initial placement
xi = x(1); yi = y(1);
hImg = imshow(sub_img, 'XData', [xi - sub_w/2, xi + sub_w/2], ...
                    'YData', [yi - sub_h/2, yi + sub_h/2]);

if hasAlpha
      set(hImg, 'AlphaData', alpha);
end
plot(x(end), y(end), 'x', 'MarkerSize', 20, 'LineWidth', 2, 'Color', 'r');

% Animate
for i = 1:length(x)
      xi = x(i);
      yi = y(i);
      thetai = theta(i);  % Radians

      % Rotate image
      rotated_img = imrotate(sub_img, -(thetai), 'nearest', 'crop');
      if hasAlpha
      rotated_alpha = imrotate(alpha, -(thetai), 'nearest', 'crop');
      set(hImg, 'AlphaData', rotated_alpha);
      end

      % Update image
      set(hImg, 'CData', rotated_img);
      set(hImg, 'XData', [xi - sub_w/2, xi + sub_w/2]);
      set(hImg, 'YData', [yi - sub_h/2, yi + sub_h/2]);

      % Update trail
      % set(trail, 'XData', x(1:i), 'YData', y(1:i));

      drawnow;
      pause(0.01);
      frame = getframe(gcf);     % or better: getframe(fig) if using a figure handle
      writeVideo(video, frame);
end
close(video);
```

APPENDIX B3: MATLAB CODE – PID CONTROLLER

```
clear all
clc
rho = 1000;         % Density of water [kg/m^3]
g = 8.91;           % Gravity [m/s^2]
D = 2.8;            % Submarine width [m]
L = 7;              % Submarine length [m]
m = 10000;          % Mass [kg]
I = (1/12) * m * L^2;  % Moment of inertia [kg*m^2]
l = 3;              % Lever arm [m]
Cd = 0.2;           % Drag coefficient
% Derived Damping Coefficients
bx = 0.5 * Cd * (pi * D^2 / 4) * rho;
by = 0.5 * Cd * (D * L) * rho;
bth = 10000;        % Rotational damping [Ns/rad]
x1e = 1;
x2e = 1;
x3e = 1;
x4e = 1;
x5e = 1;
x6e = 1;
V = L * (pi * D^2 / 4);     % Submarine volume [m^3]
Vt = V / 10;                 % Volume of each ballast tank [m^3]
Vgam = rho * g * Vt;        % Buoyancy force from one tank [N]
%inputs to linearize about
%Pat = 0;
%Pft = 0;
%Ft =  0;
%if we linearize about maximum inputs
Pat = -0.5;
Pft = -0.5;
Ft =  1385.4;
%if we linearize about half of maximum inputs
% Pat = -0.5/2;
% Pft = -0.5/2;
% Ft =  1385.4/2;
A = [0 0 0 1 0 0;
   0 0 0 0 1 0;
   0 0 0 0 0 1;
   0 0 -(1/m)*Ft*sin(x3e) -(2/m)*bx*abs(x4e) 0 0;
   0 0 (1/m)*Ft*cos(x3e) 0 -(2/m)*by*abs(x5e) 0;
   0 0 (-1/I)*(-Pat + Pft)*Vgam*l*sin(x3e) 0 0 (-2/I)*bth*abs(x6e)];


B = [0 0 0;
   0 0 0;
   0 0 0;
   0 0 (1/m)*cos(x3e);
   (1/m)*Vgam (1/m)*Vgam (1/m)*sin(x3e);
   (-1/I)*Vgam*l*cos(x3e) (1/I)*Vgam*l*cos(x3e) 0];
% Output matrices for transfer functions
```

```
C_x = [1 0 0 0 0 0];           % Output is x position
C_y = [0 0 1 0 0 0];           % Output is z position
C_theta = [0 0 0 0 1 0];       % Output is theta
D = 0;
% Simplified input definitions using index names
u_Ft     = 1;   % Forward thrust input index
u_Pat    = 2;   % Ballast tank A
u_Pft    = 3;   % Ballast tank F
% Inputs for transfer functions using logical indexing
B_x = B(:, u_Ft);                     % x controlled by F_t
B_y = B(:, [u_Pat, u_Pft]) * [1; 1];  % y controlled by P_at + P_ft
B_theta = B(:, [u_Pat, u_Pft]) * [-1; 1];  % theta controlled by P_ft - P_at
% Transfer Functions
sys_x = ss(A, B_x, C_x, D);
tf_x = tf(sys_x);
sys_y = ss(A, B_y, C_y, D);
tf_y = tf(sys_y);
sys_theta = ss(A, B_theta, C_theta, D);
tf_theta = tf(sys_theta);
%Transfer function in the laplace domain
s = tf('s');
% Ziegler-Nichols method placeholders (to be determined experimentally)
Ku_x = 20; Tu_x = 3.5;  % Example values for x
Ku_y = 30; Tu_y = 2.8;  % Example values for y
Ku_th = 18; Tu_th = 4.0; % Example values for theta
% PID gains for x (forward position)
Kp_x = 0.6 * Ku_x;
Ki_x = 2 * Kp_x / Tu_x;
Kd_x = Kp_x * Tu_x / 8;
PID_x = Kp_x + Ki_x/s + Kd_x*s;
% PID gains for y (depth)
Kp_y = 0.6 * Ku_y;
Ki_y = 2 * Kp_y / Tu_y;
Kd_y = Kp_y * Tu_y / 8;
PID_y = Kp_y + Ki_y/s + Kd_y*s;
% PID gains for theta (rotation)
Kp_th = 0.6 * Ku_th;
Ki_th = 2 * Kp_th / Tu_th;
Kd_th = Kp_th * Tu_th / 8;
PID_theta = Kp_th + Ki_th/s + Kd_th*s;
% Open-loop systems with PID controllers
open_x = PID_x * tf_x;
open_y = PID_y * tf_y;
open_theta = PID_theta * tf_theta;
% Closed-loop systems (unity feedback)
closed_x = feedback(open_x, 1);
closed_y = feedback(open_y, 1);
closed_theta = feedback(open_theta, 1);
%%  Display gains (for reference/report)
fprintf('--- PID Gains ---\n');
fprintf('x:     Kp = %.2f, Ki = %.2f, Kd = %.2f\n', Kp_x, Ki_x, Kd_x);
fprintf('y:     Kp = %.2f, Ki = %.2f, Kd = %.2f\n', Kp_y, Ki_y, Kd_y);
fprintf('theta: Kp = %.2f, Ki = %.2f, Kd = %.2f\n', Kp_th, Ki_th, Kd_th);
%% Launch PID Tuner GUIs
% These help fine-tune PID gains using MATLAB's interactive tools
pidTuner(tf_x)        % for x-position
pidTuner(tf_y)        % for y-position
```

```
pidTuner(tf_theta)    % for theta (rotation)
saveas(gcf, 'PID_Response.png');
```